

IEPS — Assignment 3 Report

Retrieval-Augmented Generation over GitHub READMEs

Tilen Lampret Pika Križnar Grega Radež
Faculty of Computer and Information Science, Ljubljana

May 2026

Abstract

We built a Retrieval-Augmented Generation (RAG) system on top of the PA2 GitHub README corpus. A query is embedded with MiniLM-L6-v2, matched in PostgreSQL/pgvector (cosine HNSW), reranked with a cross-encoder, and passed as numbered evidence to a local Ollama `llama3.2:1b` model orchestrated by `dspy`. The same queries are also answered in an LLM-only mode without retrieval. We evaluated nine queries (six README-grounded, three intentionally out-of-scope) and logged full retrieval traces before generation. When retrieval surfaces on-topic install, training, or evaluation sections, RAG answers are more repository-specific and diagnosable than LLM-only baselines. Failures remain visible: bibliography-dominated retrieval on poor queries, generation that ignores strong evidence (e.g., skipping `evaluate.py` commands), and hallucinated facts even when the prompt restricts answers to context.

Keywords: retrieval-augmented generation, pgvector, cross-encoder reranking, Ollama, explainability, GitHub README corpus

1 Introduction

In PA1 we crawled image-segmentation repositories; in PA2 we extracted README text, segmented it, embedded segments, and demonstrated semantic retrieval. PA3 extends that retrieval stack with a local large language model capable of answering natural-language questions in two different operating modes. The goal of the assignment was not only to generate answers, but also to understand how retrieval affects answer quality, transparency, and trustworthiness.

The implemented system therefore supports the following two modes:

- **With Context (RAG):** retrieve the top relevant segments from the corpus, format them as explicit evidence, and generate an answer conditioned on that evidence.
- **Without Context (LLM-only):** answer the question directly using only the language model without retrieval support.

The assignment strongly emphasized explainability. Because of this, our implementation was designed so that all retrieved evidence is visible before generation. The user can inspect the retrieved chunks, see their ranking scores, inspect repository URLs, and determine whether the evidence itself is reliable before trusting the generated answer.

Compared to a standard chatbot pipeline, this architecture makes debugging significantly easier. If the final answer is incorrect, we can separate the problem into two categories:

1. retrieval failed and returned poor or irrelevant evidence,
2. generation failed despite having good evidence available.

This distinction became especially important during evaluation because several answers failed for very different reasons. Some failures happened because the requested information did not exist in the README corpus, while others happened because the small local LLM hallucinated unsupported details even when the retrieval stage returned strong evidence.

This report therefore focuses on three major aspects required by the assignment:

- implementation decisions and system architecture,
- evaluation of chunking and retrieval quality,
- analysis of explainability, hallucination behavior, and retrieval effectiveness.

2 Implementation

2.1 Overview

PA3 extends PA2 with answer generation modules located in `pa3/rag/`. The retrieval implementation itself was intentionally reused instead of rewritten. The file `pa3/rag/pa2_retrieval.py` dynamically imports the retrieval implementation from PA2, ensuring that the same embedding pipeline, vector database structure, and similarity logic are preserved across assignments.

The overall execution flow is implemented inside `answer_query()` in `pipeline.py`. Depending on the selected mode, the pipeline either:

1. retrieves evidence chunks,
2. formats the context,
3. constructs the final prompt,
4. sends the request to the local LLM,
5. stores and prints the results.

In LLM-only mode, the retrieval stage is skipped entirely and the query is directly forwarded to the model with a simplified prompt.

The separation between retrieval and generation proved useful during testing because it allowed us to independently evaluate vector search quality and generation quality. This also simplified experimentation with reranking, top-k retrieval, and prompt formatting.

2.2 Corpus and document structure (from PA2)

Documents are GitHub README pages represented as chunk-level evidence. The corpus originates from the PA1 crawl of image-segmentation repositories and was transformed into a searchable retrieval dataset during PA2.

The preprocessing pipeline consisted of three major stages:

1. **Extraction:** HTML pages were cleaned and transformed into structured text. Only the markdown-body subtree was preserved. Heading markers such as [H1]–[H6] were inserted to preserve document hierarchy.
2. **Segmentation:** extracted README content was split into semantically meaningful chunks using the `heading_structure_v4` strategy. The segmentation algorithm attempted to preserve section coherence while keeping chunk sizes within the embedding model context budget.
3. **Embedding:** each chunk was embedded using MiniLM and stored in PostgreSQL together with metadata such as repository URL, page ID, segment ID, and raw text.

Instead of splitting documents using only fixed token windows, the segmentation strategy tried to preserve README semantics. Installation sections, evaluation commands, training instructions, and dataset descriptions therefore usually remained grouped together. This significantly improved interpretability during retrieval because retrieved chunks often corresponded to meaningful README subsections rather than arbitrary text fragments.

The chosen segmentation approach also improved explainability. When a generated answer referenced a retrieved chunk, the user could directly inspect the original README section and understand where the information came from.

2.3 Chunking strategy and trade-offs

The chunking strategy was one of the most important implementation decisions because chunk boundaries directly affect retrieval quality.

Our implementation used heading-aware chunking with approximate token targets between 180 and 320 tokens and a hard cap around 420 tokens. This approach attempted to balance two competing goals:

- **Smaller chunks:** improve retrieval granularity because the retrieved text is usually more focused and specific.
- **Larger chunks:** preserve context continuity and reduce the risk of separating related information.

During PA2 segmentation design and inspection of retrieved chunks in PA3, we observed several important trade-offs:

- Very small chunks frequently lost important surrounding context. Commands, explanations, and dataset names could become separated across multiple chunks.
- Very large chunks often reduced retrieval precision because multiple unrelated README sections became merged together.
- Bibliography sections sometimes became highly ranked because they contained many technical keywords even though they were not useful for answering the query.

The heading-aware approach performed better than naive fixed-window splitting because README files naturally contain semantically organized sections. Preserving this structure made retrieval results easier to interpret and generally improved answer quality in RAG mode.

2.4 Embeddings and vector retrieval

Failure type	Symptom	Example queries
<code>irrelevant_retrieval</code>	High scores for bibliography/reference sections	poor_1
<code>insufficient_coverage</code>	Topic absent from README crawl	poor_1, poor_2
<code>model_hallucination</code>	Claims not present in retrieved chunks	good_2 (extra datasets), good_3, poor_3
<code>generative_without_evidence</code>	Plausible but unsupported LLM-only text	good_2, poor_1–poor_3

Table 1: Observed failure modes during evaluation.

The crawled corpus mainly consists of English technical README documentation. Because of this, MiniLM represented a good balance between embedding quality, computational cost, and retrieval speed.

The same embedding model was already used in PA2, which simplified compatibility with the existing pgvector index. Reusing the same embedding space also allowed direct comparison between retrieval-only experiments from PA2 and the RAG pipeline from PA3.

Cosine similarity was used as the primary retrieval metric because embeddings were normalized before storage. The pgvector HNSW index enabled efficient approximate nearest-neighbor retrieval even with tens of thousands of segments.

Although larger embedding models may produce better semantic representations, MiniLM was selected because the assignment emphasized local execution and reproducibility on standard hardware.

2.5 Reranking

First-stage vector search retrieves `candidate_k=20` segments. These results are then reranked using the cross-encoder model `cross-encoder/ms-marco-MiniLM-L-6-v2`. After reranking, only the top-5 segments are preserved as final evidence.

The reranking stage was introduced because pure cosine similarity often overemphasized keyword overlap. In technical README corpora this can produce problematic retrieval behavior. Bibliography sections, benchmark tables, or unrelated repositories may contain many overlapping technical terms even when they are not useful for answering the query.

The cross-encoder improves ranking quality because it jointly processes the query and passage instead of comparing only embedding vectors.

We compared vector-only retrieval and reranked retrieval on two representative examples:

The reranking stage noticeably improved several good-query results, especially queries related to training commands, evaluation scripts, or dependency installation. However, reranking could not solve cases where the corpus itself lacked the requested information.

2.6 Language model and orchestration

Generation is handled through Ollama running locally on the machine. The chosen model was `llama3.2:1b`, a relatively small local language model with approximately 1.3 billion parameters.

Query	Mode	Rank-1 segment (URL; <code>segment_id</code>)
good_1 (mIoU eval)	No rerank	FCN-Segmentation-TensorFlow; seg. 743653
good_1	With rerank	OCNet; seg. 755357 (validation mIoU text)
poor_1 (medical 2026)	No rerank	3D MRI brain tumour (Medical Decathlon); seg. 754181
poor_1	With rerank	TF-1D-2D pipelines bibliography; seg. 741817

Table 2: Reranking changes the top hit: for good_1 it promotes OCNet evaluation content over a generic FCN repository; for poor_1 both top results remain weak, but reranking still changes ordering behavior.

Property	Value
Runtime	Ollama (local HTTP API, <code>localhost:11434</code>)
Model	<code>llama3.2:1b</code> via <code>dspy</code> (<code>ollama_chat/llama3.2:1b</code>)
Parameters	~ 1.3 billion
Preflight	<code>ollama_health.require_ollama()</code> before generation

Table 3: LLM configuration.

The model choice was mainly motivated by:

- reproducibility on standard laptops,
- low memory requirements,
- fast local inference,
- compatibility with the assignment requirements.

The pipeline uses `dspy` signatures for structured prompting through `Predict(SimpleRAG)` and `Predict(SimpleQA)`. Batch evaluation used signatures by default (`use_signature=true`); the CLI also supports `-plain-prompt` for raw template strings, but we did not run a separate comparison experiment.

Despite these advantages, the small model size also introduced important limitations. The model occasionally ignored explicit prompt instructions, hallucinated unsupported facts, or generated additional information not present in the retrieved evidence.

These limitations became especially visible during poor-query evaluation and represent one of the major observations of the assignment.

2.7 Prompt construction and context assembly

RAG and LLM-only templates are defined in `pa3/rag/prompts.py`.

The RAG prompt explicitly instructs the model to:

- answer using only the provided context,
- avoid unsupported claims,
- cite relevant chunk numbers,
- explicitly state when the context is insufficient.

The LLM-only prompt is intentionally simpler and only discourages repository-specific hallucinations.

Retrieved chunks are formatted as numbered evidence blocks containing:

- retrieval rank,
- repository URL,
- segment ID,
- page ID,
- cosine distance or cross-encoder rerank score (depending on mode),
- full segment text.

This structure makes the retrieval stage transparent and easy to inspect.

Context assembly is handled by `format_context()`. The final context is capped at `max_context_chars=7000`. If the context becomes too long, lower-ranked chunks are removed first.

This design choice introduced another trade-off:

- larger contexts increase available evidence,
- smaller contexts reduce token usage and generation noise.

In practice, the 7000-character limit worked reasonably well for README-style technical documentation.

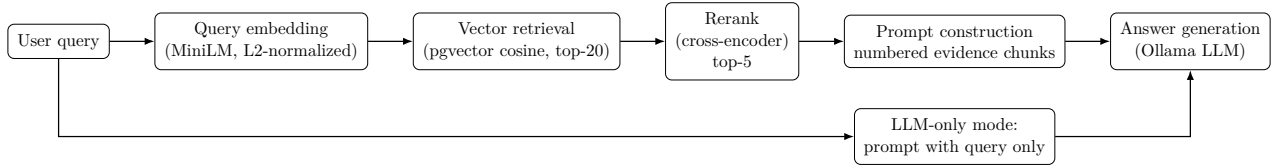


Figure 1: RAG (With Context) and LLM-only (Without Context) pipelines. Hits are logged before generation. The rerank stage is omitted when `-no-rerank` is set.

Default configuration: `top_k=5`, `candidate_k=20`, reranking enabled, `max_context_chars=7000`.

2.8 Explainability mechanisms

One of the most important goals of the project was explainability. Because of this, the system was designed to expose internal retrieval behavior rather than hide it behind the final answer.

The following explainability mechanisms were implemented:

- **CLI tracing:** the command-line interface prints retrieved chunks, scores, URLs, and segment IDs before generation.
- **JSON logging:** complete retrieval traces and generated outputs are stored in `RunResult` JSON structures.
- **Batch evaluation logging:** `eval/run_eval.py` stores retrieval evidence, prompts, context, and answers for all evaluation queries.
- **Failure diagnosis:** retrieval quality and generation quality can be analyzed independently.

This architecture proved extremely useful during evaluation because it allowed us to inspect exactly why a generated answer failed.

For example, if the top retrieved chunks are unrelated bibliography entries, then the retrieval stage is responsible. However, if the retrieved chunks clearly contain the correct answer but the model still invents unsupported details, then the problem originates from generation.

Without explicit retrieval logging, these two failure categories would be difficult to distinguish.

2.9 Libraries and tools

Component	Library / tool
Database	PostgreSQL 17 + pgvector
DB driver	psycopg, pgvector
Bi-encoder / reranker	sentence-transformers, PyTorch
LLM orchestration	dspy
Local inference	Ollama
Evaluation config	PyYAML
Data handling	Python standard library + JSON

Table 4: Main tools used in the implementation.

Each tool served a different role inside the pipeline. PostgreSQL and pgvector provided efficient vector search capabilities, sentence-transformers handled embedding and reranking models, and Ollama enabled fully local inference without external APIs.

The use of local models also simplified reproducibility because the complete pipeline can run offline once the required models are downloaded.

3 System evaluation criteria

3.1 Data processing evaluation

The quality of the data processing pipeline was evaluated primarily through retrieval relevance and chunk interpretability.

Because the assignment focused on explainability, it was important that retrieved chunks remain understandable to humans. During evaluation we therefore manually inspected whether retrieved chunks:

- preserved semantic coherence,
- contained enough context to answer the query,
- avoided splitting commands and explanations,
- remained small enough for precise retrieval.

The heading-aware chunking strategy generally performed well because README files naturally contain structured sections such as:

- installation,
- training,
- evaluation,
- dataset preparation,
- pretrained model usage.

This structure improved retrieval granularity while preserving readability.

However, several weaknesses were also observed:

- bibliography-heavy sections sometimes ranked highly,
- some merged sections became too broad,
- missing README information cannot be recovered by retrieval.

The final segmentation strategy represented a compromise between retrieval precision and contextual completeness.

3.2 Data retrieval evaluation

Setting	Value
First-stage retrieval	Cosine similarity, HNSW index
Top- k returned	5
Candidate pool	20
Reranker	<code>cross-encoder/ms-marco-MiniLM-L-6-v2</code>

Table 5: Retrieval settings used during evaluation.

Retrieval quality was evaluated manually by inspecting whether the returned chunks actually contained information necessary for answering the query.

A retrieved chunk was considered:

- **relevant** if it directly contained commands, metrics, package names, training instructions, or explanations required for the answer,
- **partially relevant** if it belonged to the correct repository but lacked direct answer content,
- **irrelevant** if it consisted mostly of bibliography entries, unrelated repositories, or tangential benchmark discussions.

We also analyzed the effect of top-k selection on answer quality.

Using only the top-1 result frequently reduced context coverage too aggressively. On the other hand, very large top-k values introduced noisy evidence and increased the probability of hallucination.

The selected configuration of top-5 reranked chunks represented a practical compromise between evidence diversity and retrieval precision.

The retrieval stage had a direct impact on answer quality:

- strong retrieval usually improved specificity and explainability,
- weak retrieval often caused vague or hallucinated generation,
- missing corpus coverage could not be solved by prompting alone.

3.3 Generation evaluation criteria

Generation quality was evaluated qualitatively rather than through automatic metrics.

The following aspects were analyzed:

- factual consistency with retrieved evidence,
- repository specificity,
- command correctness,
- hallucination frequency,
- transparency and explainability.

Particular attention was given to cases where the generated answer contained unsupported claims not present in the retrieved evidence.

The assignment specifically emphasized explainability, so we considered an answer more trustworthy when:

- retrieved evidence clearly supported the answer,
- the answer remained close to the retrieved content,
- hallucinated additions were minimized,
- the user could verify claims directly from chunks.

LLM-only answers were usually less explainable because they lacked explicit supporting evidence.

3.4 Evaluation protocol

The evaluation set consisted of nine total queries:

- six README-grounded queries expected to succeed,
- three intentionally difficult or out-of-scope queries.

The original PA2 demo queries were reused and extended with additional repository-specific tasks involving:

- dependency installation,
- pretrained weights,
- custom training.

Evaluation was executed using:

```
cd pa3/rag
python eval/run_eval.py --output ../output/eval_run.json
```

Configuration snapshot (2026-05-26):

- top_k=5
- candidate_k=20
- reranking enabled
- llama3.2:1b

Full retrieval traces and generated outputs were stored in `pa3/output/eval_run.json` and summarized in `pa3/output/eval_run.md`. Reranking impact was analyzed separately using vector-only vs. reranked retrieval on the same database snapshot (Table 2).

4 Results and analysis

4.1 Query design

The evaluation queries were intentionally designed to cover both realistic success cases and realistic failure cases.

Good queries (good_1–good_6) focus on information commonly available inside technical README documentation:

- evaluation commands,
- dataset names,
- installation dependencies,
- pretrained checkpoints,
- training instructions.

Poor queries (poor_1–poor_3) intentionally require information that static GitHub READMEs cannot reliably provide:

- best medical model in 2026,
- EU hospital procurement advice,
- recent benchmark comparisons.

These queries were included because they test whether the system can appropriately fail or acknowledge insufficient evidence.

4.2 Comparison across nine queries

Table 6 summarizes retrieval behavior, generated answers, and overall evaluation.

Table 6: Evaluation comparison: With Context (RAG) vs. Without Context (LLM-only).

ID	Query (short)	Top retrieval	RAG answer (summary)	LLM-only (summary)	Verdict
good_1	Evaluate mIoU on validation set	OCNet, DeepLab-v3 (validation commands)	Hyperparameter and OHEM explanation	Generic precision/recall explanation	RAG better
good_2	Food classification repositories	Food-Categories, Food-Classification-DL	Lists datasets and repositories	Generic food classification statement	RAG better (minor hallucination)
good_3	SAM inference / checkpoints	SAM-HQ checkpoint usage	Grounded checkpoints; invented <code>list_models()</code> API	Invented repository URL	Mixed
good_4	Python dependencies	<code>pip install segmentation_pipeline</code>	Concrete package names	Generic Python packages	RAG better
good_5	Download pretrained weights	contrastive-unpaired-translation	Concrete download guidance	Generic model hub answer	RAG better
good_6	Train on custom dataset	ResNet-50-CBAM, Mask R-CNN	Correct dataset/config workflow	Generic ML checklist	RAG better
poor_1	Best medical model 2026	Bibliography-heavy retrieval	Weak fragmented answer	Generic medical AI discussion	Both weak
poor_2	Hospital EU regulatory purchase	Polyp benchmark text	Irrelevant architecture fragment	Speculative regulatory advice	Both weak
poor_3	Exact mIoU vs Mask2Former	Benchmark citation blocks	Unsupported metric comparison	Fully hallucinated metrics	Both weak

4.3 Good queries: detailed analysis

For good queries, retrieval generally returned semantically relevant README sections. In these cases, the RAG pipeline consistently produced more repository-specific and more explainable answers than the LLM-only baseline.

The largest improvements appeared in queries involving:

- installation instructions,
- command-line usage,

- pretrained checkpoint downloads,
- repository-specific APIs.

The retrieval stage helped anchor the model to concrete evidence instead of generic prior knowledge.

good_1 — mIoU evaluation query. The top retrieved chunks contained validation commands and evaluation scripts from OCNet and DeepLab repositories.

The RAG answer correctly discussed OHEM hyperparameters and validation procedures. However, the model still underused some of the strongest retrieved evidence and failed to directly restate several evaluation commands present in the chunks.

The LLM-only answer instead generated a generic explanation of precision, recall, and sklearn metrics unrelated to the actual repositories.

This example demonstrates that retrieval significantly improved relevance even though generation quality was still imperfect.

good_2 — food classification repositories. Retrieval correctly identified food-classification repositories and related dataset discussions.

The RAG answer produced repository-specific output and referenced realistic datasets. Food-101 appeared in retrieved chunk `segment_id` 754520, but Flickr34, YFCC-100M, and Stanford Food Image Dataset were **not** present in any top-5 `segment_text` (partial hallucination on otherwise good retrieval).

The LLM-only answer remained generic and lacked repository grounding.

This example demonstrates an important observation from the assignment: retrieval can improve relevance while hallucinations still remain possible.

good_3 — SAM inference and checkpoints. Retrieval returned strong SAM-HQ evidence containing checkpoint download instructions and `SamPredictor` usage examples.

The RAG answer was mostly grounded in the retrieved evidence but still invented a non-existent `list_models()` API.

The LLM-only answer hallucinated an entirely fake repository URL.

This example highlights the difference between partially grounded hallucinations and fully unsupported hallucinations.

good_4 — dependency installation. The retrieved evidence directly contained dependency installation commands and package lists.

The RAG answer accurately listed relevant packages such as `musket-ml`, `pytorch`, `torchvision`, `numpy`, and `OpenCV`.

The LLM-only answer only produced vague references to generic Python dependencies.

This was one of the clearest examples where retrieval strongly improved both answer quality and explainability.

good_5 — pretrained weights. The retrieval stage returned chunks discussing pretrained checkpoint downloads and repository hosting.

The generated RAG answer remained relatively close to the retrieved evidence and provided useful repository-specific guidance.

The LLM-only answer instead described generic Hugging Face or Keras workflows unrelated to the corpus.

good_6 — custom dataset training. Retrieved chunks contained dataset folder structures and subclassing examples for custom training.

The RAG answer mirrored the retrieved configuration workflow and remained relatively grounded in the evidence.

The LLM-only answer again defaulted to a generic machine-learning training checklist.

4.4 Poor queries: detailed analysis

The intentionally difficult queries revealed several important limitations of the system.

poor_1 — best medical model in 2026. The retrieval stage returned bibliography-heavy medical segmentation references instead of direct ranking information.

The generated RAG answer became fragmented and incomplete because the evidence itself was insufficient.

The LLM-only answer generated a plausible-sounding medical AI explanation but without reliable support.

This example demonstrates that retrieval quality fundamentally bounds generation quality.

poor_2 — EU hospital procurement advice. The retrieval stage returned benchmark and architecture descriptions unrelated to procurement or regulation.

The generated answer therefore became meaningless and disconnected from the original question.

The LLM-only answer produced speculative statements about regulations and U-Net architectures despite lacking evidence.

This query demonstrated the limitations of the README-only corpus.

poor_3 — exact benchmark comparison. Top-ranked chunks were Mask2Former citation blocks without Cityscapes scores. A lower-ranked SeMask segment (`segment_id` 745934) contained a 57.52 mIoU value, but nothing in the evidence supported a time-bound comparison to Mask2Former “last month.”

The RAG answer nevertheless claimed an unsupported SeMask vs. Mask2Former comparison. The LLM-only answer hallucinated even larger benchmark values (“~95% vs. ~92%”).

This example demonstrates that prompt instructions alone are insufficient to fully prevent hallucinations in smaller local language models.

4.5 When retrieval improves answer quality

Retrieval improved answer quality most strongly when:

- the answer existed directly inside README documentation,
- retrieved chunks contained commands or configuration examples,
- repository-specific terminology was required,
- the query closely matched README semantics.

The largest improvements were observed in:

- installation queries,
- dependency queries,
- pretrained checkpoint questions,
- custom training workflows.

In these cases, retrieval significantly increased transparency because users could directly inspect the supporting evidence.

4.6 When retrieval fails

Retrieval performed poorly when:

- the corpus lacked the requested information,
- queries required recent external knowledge,
- bibliography sections dominated cosine similarity,
- repository README content was too sparse.

Even with reranking enabled, the system cannot generate reliable answers if the underlying corpus does not contain relevant evidence.

This became especially visible for regulatory, medical, and time-sensitive benchmark questions.

4.7 Failure taxonomy and diagnosability

Failure type	Symptom	Example queries
<code>irrelevant_retrieval</code>	High scores for bibliography/reference sections	poor_1
<code>insufficient_coverage</code>	Topic absent from README crawl	poor_1, poor_2
<code>model_hallucination</code>	Claims not present in retrieved chunks	good_2 (extra datasets), good_3, poor_3
<code>generative_without_evidence</code>	Plausible but unsupported LLM-only text	good_2, poor_1–poor_3

Table 7: Observed failure modes during evaluation.

One of the major strengths of the RAG architecture is diagnosability.

Because retrieval evidence is explicitly logged, users can inspect whether the system failure originated from:

- retrieval,
- corpus coverage,
- generation behavior.

This transparency is significantly harder to achieve in pure LLM-only systems.

In several evaluation cases, the LLM-only answers appeared more fluent than the RAG answers even though they were less trustworthy. This is an important practical observation because users may incorrectly trust fluent unsupported answers.

The RAG pipeline therefore improves not only answer specificity but also transparency and debuggability.

5 Limitations and future work

Several limitations became visible during development and evaluation.

- **README-only corpus:** the dataset does not include GitHub issues, papers, discussions, or external web sources.
- **Small local model:** `llama3.2:1b` occasionally ignored instructions and hallucinated unsupported details.
- **Reranking limitations:** reranking improves ordering quality but cannot recover missing information.
- **Context truncation:** lower-ranked but useful chunks may be removed when the context becomes too large.
- **No citation verification:** the system does not automatically verify whether generated claims actually appear inside retrieved evidence.
- **Alternative LLMs not used:** the course brief mentions larger or multilingual local models (e.g. GaMS, Salamandra); we stayed with `llama3.2:1b` for speed and reproducibility.

Future improvements could include:

- larger local LLMs,
- explicit hallucination detection,
- stricter citation enforcement,
- hybrid web retrieval,
- automatic confidence estimation,
- retrieval score thresholding.

A larger evaluation benchmark with human scoring could also provide more rigorous quantitative comparison between RAG and LLM-only modes.

6 Conclusion

This project connected the semantic retrieval pipeline from PA2 with a local Retrieval-Augmented Generation architecture capable of transparent evidence-based answering.

The evaluation demonstrated that retrieval substantially improves answer specificity, repository grounding, and explainability for README-based technical questions. RAG answers were generally more informative and easier to verify than LLM-only outputs.

At the same time, the experiments also showed important limitations:

- retrieval quality fundamentally bounds generation quality,
- missing corpus coverage cannot be solved by prompting,
- small local LLMs still hallucinate unsupported details,
- fluent answers are not necessarily trustworthy.

The most important insight from the assignment was therefore not only how to build a RAG system, but also how retrieval evidence improves transparency and diagnosability.

Even when the generated answer is incorrect, explicit retrieval logging allows users to understand why the system failed. This makes RAG systems significantly easier to debug, evaluate, and trust compared to opaque LLM-only pipelines.

References

- PA2 pipeline: `pa2/README.md`
- Course RAG demo: `assignment-2-notebooks/vaje5-RAG_simple_demo.ipynb`
- PA3 usage and reproduction: `pa3/README.md`
- Evaluation artifacts: `pa3/output/eval_run.json`; rerank ablation in Table 2 (vector-only vs. reranked top-1 on the same DB snapshot)